

École Polytechnique de Ouagadougou

IGIT 2 : Génie Informatique

Module :

Ingénierie de la cryptographie

Cryptologie, clés et certificats

Rapport de travaux pratiques

TP 1 : Chiffrement authentifié et dérivation de clés
TP 2 : Construire une infrastructure de gestion de clés
TP 3 : Déployer un canal sécurisé et gérer les certificats

Membre :

- TRAORE Inoussa

Enseignant :

Dr. Zakaria SAWADOGO

Année académique 2026 – 2027

Table des matières

Présentation générale 3

I TP 1 – Chiffrement authentifié et dérivation de clés 4

1 Objectifs et conception 4
1.1 Format du fichier chiffré (versionné) 4
1.2 Chaîne cryptographique 4

2 Étape 1 – Dérivation des clés 4

3 Étape 2 – Chiffrer et déchiffrer 5

4 Étape 3 – Tests automatiques 5

5 Étape 4 – Évolution du format 6

6 Questions de réflexion 7

7 Bilan du TP 1 7

II TP 2 – Construire une infrastructure de gestion de clés 8

1 Objectifs et architecture 8

2 Étape 1 – Les autorités et leurs fichiers de configuration 8

3 Étape 2 – AC racine 9

4 Étape 3 – AC intermédiaire 9

5 Étape 4 – Certificats de serveur et de client 9

6 Étape 5 – Validation 10

7 Étape 6 – Révocation 10

8 Étape 7 – Script `pki.sh` 10

9 Questions de réflexion 11

10 Bilan du TP 2 12

III TP 3 – Déployer un canal sécurisé et gérer les certificats 13

1 Objectifs et points d'attention 13

2 Étape 1 – Préparation des fichiers 13

3 Étape 2 – Configuration de nginx 13

4 Étape 3 – Observation de la poignée de main **14**
4.1 Connexions testées . . . 14
4.2 Paramètres de la connexion TLS . . . 14
4.3 Analyse des messages de la poignée de main . . . 14

5 Étape 4 – Vérification de la configuration **15**

6 Étape 5 – Supervision : `surveiller.py` **15**

7 Étape 6 – Renouvellement : `renouveler.sh` **16**

8 Questions de réflexion **17**

9 Bilan du TP 3 **18**

Conclusion générale **19**

Annexes **20**

A TP 1 – Code source **20**
A.1 `coffre.py` . . . 20
A.2 `test_coffre.py` . . . 21
A.3 `demo.py` . . . 22

B TP 2 – Fichiers de configuration **22**
B.1 `racine.cnf` . . . 22
B.2 `inter.cnf` . . . 23

C TP 2 – Script `pki.sh` **23**

D TP 3 – Configuration nginx **24**

E TP 3 – Script `surveiller.py` **25**

F TP 3 – Script `renouveler.sh` **26**

Présentation générale

Les trois séances suivent la progression du cours. Le TP 1 construit un module de chiffrement authentifié protégé par mot de passe. Le TP 2 construit une infrastructure à clé publique (racine, intermédiaire, certificats de serveur et de client, révocation). Le TP 3 déploie un serveur TLS avec authentification mutuelle, puis supervise et renouvelle les certificats en s'appuyant sur la PKI du TP 2. Dans les trois cas aucune primitive n'est réimplémentée : on assemble des briques éprouvées, on les teste et on les exploite.

Environnement de travail

Élément	Valeur
Système	Ubuntu 24.04.4 LTS (conteneur Linux), 1 cœur, 4 Go de mémoire
Python et bibliothèques	Python 3.12.3 ; `cryptography` 46.0.6 ; `argon2-cffi` 25.1.0 ; `pytest` 9.1.1
Outils TLS	OpenSSL 3.0.13 ; nginx 1.24.0 ; curl ; nmap 7.94
Règles respectées	aucune primitive réimplémentée ; aucune clé privée dans le compte rendu ; manipulations limitées aux noms `*.tp.local` et à la machine locale

Les durées mesurées dépendent de la machine ; elles sont données pour l'environnement ci-dessus.

Première partie

TP 1 – Chiffrement authentifié et dérivation de clés

1 Objectifs et conception

Le but est un mini-coffre de fichiers, `coffre.py`, qui chiffre un fichier avec un mot de passe, et dont on démontre par des tests automatiques la confidentialité et l'intégrité.

1.1 Format du fichier chiffré (versionné)

Champ	Taille	Rôle
Identifiant `CFR1`	4 octets	reconnaître le format et sa version
Paramètres Argon2id (t, m, p)	12 octets	trois entiers de 32 bits (`>III`) pour redériver la clé et durcir plus tard
Sel	16 octets	rendre la dérivation unique pour ce fichier
Nonce	12 octets	unique pour ce chiffrement
Chiffré + étiquette	variable	AES-256-GCM (étiquette de 16 octets)

L'en-tête (44 octets) suivi du nom du fichier constitue les **données associées** : authentifiées, non chiffrées.

1.2 Chaîne cryptographique

$$\text{mot de passe} \xrightarrow[\text{sel, } (t,m,p)]{\text{Argon2id}} K_{\text{maître}} \xrightarrow[\text{info}]{\text{HKDF-SHA256}} K_{\text{fichier}} \xrightarrow[\text{nonce, AD}]{\text{AES-256-GCM}} \text{chiffré}\|\text{étiquette}$$

2 Étape 1 – Dérivation des clés

```python
def cle_maitresse(mot_de_passe: str, sel: bytes, t=T, m=M, p=P) -> bytes:
    return hash_secret_raw(secret=mot_de_passe.encode("utf-8"), salt=sel,
                           time_cost=t, memory_cost=m, parallelism=p,
                           hash_len=32, type=Type.ID)

def sous_cle(maitresse: bytes, usage: bytes) -> bytes:
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
                info=b"coffre-v1|" + usage).derive(maitresse)
```

Argon2id (`Type.ID`) combine la résistance aux canaux auxiliaires et aux attaques parallèles sur GPU ; la sortie de 32 octets donne une clé AES-256. HKDF reçoit une valeur déjà uniforme : son rôle est la *séparation des usages* par le paramètre `info` (une même clé ne sert jamais à deux usages).

Mesure du temps de dérivation (moyenne de trois essais, $p = 4$) :

t	m (Mio)	Durée
3	64	0,22 s
2	128	0,32 s
4	128	0,58 s
3	128	0,60 s

Paramètres retenus : $t = 3$, $m = 128$ Mio, $p = 4$ (la constante `M` du module est portée à `128 * 1024`), soit environ 0,5 à 0,6 s par dérivation. On a augmenté d'abord la mémoire, paramètre le plus coûteux pour un attaquant utilisant du matériel massivement parallèle, puis le nombre de passes. Contrainte technique rencontrée : Argon2 impose $m \geq 8p$.

3 Étape 2 – Chiffrer et déchiffrer

```python
def chiffrer(mot_de_passe, nom, donnees, t=T, m=M, p=P) -> bytes:
    sel, nonce = os.urandom(LEN_SEL), os.urandom(LEN_NONCE)
    entete = MAGIC + struct.pack(">III", t, m, p) + sel + nonce
    cle = _cle_fichier(mot_de_passe, sel, t, m, p)
    ad = entete + nom.encode("utf-8")
    return entete + AESGCM(cle).encrypt(nonce, donnees, ad)

def dechiffrer(mot_de_passe, nom, blob) -> bytes:
    if len(blob) < LEN_ENTETE + LEN_TAG or blob[:4] != MAGIC:
        raise ValueError("format inconnu")
    t, m, p = struct.unpack(">III", blob[4:16])
    if not (1 <= t <= T_MAX and 8 * p <= m <= M_MAX and 1 <= p <= P_MAX):
        raise ValueError("parametres hors bornes")
    sel, nonce = blob[16:32], blob[32:44]
    cle = _cle_fichier(mot_de_passe, sel, t, m, p)
    ad = blob[:LEN_ENTETE] + nom.encode("utf-8")
    return AESGCM(cle).decrypt(nonce, blob[LEN_ENTETE:], ad)   # InvalidTag
```

Choix de conception. Le sel et le nonce sont tirés par `os.urandom` *dans* la fonction, donc neufs à chaque appel. Les données associées couvrent l'en-tête complet puis le nom du fichier. Un identifiant inconnu est rejeté avant toute dérivation coûteuse. Comme les paramètres sont lus *avant* d'être authentifiés, on les borne (mémoire ≤ 1 Gio, etc.) pour éviter qu'un fichier forgé n'épuise la mémoire (déni de service).

Démonstration (script `demo.py`, annexe A) :

```
$ python3 demo.py
clair : 990 octets ; chiffre : 1050 octets (= 990 + 44 + 16)
en-tete (hex) : 43465231 | 000000030002000000000004 | cce32f2b40c98a036820117dd1c07f83 |
    eb9faf6e47b3f2da924ad746
identifiant = b'CFR1' ; t, m, p = 3, 131072 Kio (128 Mio), 4
deux chiffrements identiques ? False ; sels egaux ? False
dechiffrement correct ? True
apres rechiffrement : t = 4
ancien mot de passe -> InvalidTag
nouveau mot de passe correct ? True
```

La taille du chiffré vaut bien clair + 44 (en-tête) + 16 (étiquette). L'en-tête se décode : `43465231` est `CFR1` ; `00000003 00020000 00000004` donne $t = 3$, $m = 131072$ Kio, $p = 4$. Deux chiffrements du même clair diffèrent, y compris par le sel.

4 Étape 3 – Tests automatiques

Les tests utilisent des paramètres légers ($t = 1$, $m = 8$ Mio) afin d'être rapides ; le déchiffrement relit les paramètres dans l'en-tête.

Test	Comportement observé	Propriété vérifiée
Aller-retour	données identiques	correction
Deux chiffrements du même fichier	deux résultats différents	chiffrement probabiliste (sel et nonce neufs)
Mauvais mot de passe	`InvalidTag`	authentification de la clé
Un octet du chiffré modifié (octet 47, puis dernier octet)	`InvalidTag`	intégrité du chiffré et de l'étiquette
Un octet du sel modifié	`InvalidTag`	authentification de l'en-tête
Fichier renommé (nom échangé)	`InvalidTag`	liaison au contexte par les données associées
Identifiant `XXXX`	`ValueError`	rejet d'un format inconnu

```python
@pytest.mark.parametrize("pos", [coffre.LEN_ENTETE + 3, -1])
def test_octet_modifie(pos):
    blob = bytearray(chif())
    blob[pos] ^= 0x01
    with pytest.raises(InvalidTag):
        coffre.dechiffrer(MDP, NOM, bytes(blob))

def test_fichier_renomme():
    with pytest.raises(InvalidTag):
        coffre.dechiffrer(MDP, "autre.txt", chif())
```

Résultat de l'exécution :

```
$ python -m pytest -v
test_coffre.py::test_aller_retour                    PASSED   [ 10%]
test_coffre.py::test_chiffrement_probabiliste        PASSED   [ 20%]
test_coffre.py::test_mauvais_mot_de_passe            PASSED   [ 30%]
test_coffre.py::test_octet_modifie[47]               PASSED   [ 40%]
test_coffre.py::test_octet_modifie[-1]               PASSED   [ 50%]
test_coffre.py::test_entete_modifie                  PASSED   [ 60%]
test_coffre.py::test_fichier_renomme                 PASSED   [ 70%]
test_coffre.py::test_format_inconnu                  PASSED   [ 80%]
test_coffre.py::test_rechiffrement                   PASSED   [ 90%]
test_coffre.py::test_anciens_parametres_lisibles     PASSED   [100%]
============================== 10 passed in 0.23s ==============================
```

Analyse. Dans tous les cas AES-GCM lève `InvalidTag` : aucune donnée corrompue n'est jamais rendue. Le test sur le dernier octet porte sur l'étiquette elle-même, celui sur l'octet 47 sur le chiffré. Au total 10 tests sont réussis (le test paramétré compte pour deux).

5 Étape 4 – Évolution du format

Le format est évolutif pour trois raisons. (1) Les paramètres (t, m, p) sont stockés dans chaque fichier et lus au déchiffrement : relever les constantes du code ne rend pas illisibles les anciens fichiers. (2) L'identifiant `CFR1` permet d'introduire un `CFR2` tout en continuant à lire `CFR1`. (3) Le sel étant propre à chaque fichier, durcir les paramètres ne le touche pas.

```python
def rechiffrer(ancien_mdp, nouveau_mdp, nom, blob, t=T, m=M, p=P) -> bytes:
    donnees = dechiffrer(ancien_mdp, nom, blob)
    return chiffrer(nouveau_mdp, nom, donnees, t, m, p)
```

Un nouveau sel et un nouveau nonce sont tirés : rien n'est réutilisé de l'ancien fichier. Vérification (voir la démonstration ci-dessus) : après `rechiffrer` avec $t = 4$, l'ancien mot de passe échoue avec `InvalidTag`, le nouveau réussit, et l'en-tête annonce bien $t = 4$. Les tests `test_rechiffrement` et `test_anciens_parametres_lisibles` confirment aussi qu'un fichier chiffré avec des paramètres plus faibles reste lisible.

6 Questions de réflexion

1. Pourquoi le sel est-il stocké en clair ? Le sel n'est pas un secret : il garantit l'unicité de la dérivation pour chaque fichier et il est indispensable au déchiffrement. Sa publication n'affaiblit pas la sécurité, car l'attaquant doit toujours deviner le mot de passe. Il empêche en revanche les tables précalculées et l'attaque simultanée de plusieurs fichiers : deux fichiers protégés par le même mot de passe ont des clés différentes.

2. Pourquoi aucun risque de répétition du nonce ? La clé K_{fichier} dépend d'un sel aléatoire de 128 bits : chaque clé ne chiffre qu'une fois, donc avec un seul nonce. Une répétition ne serait dangereuse que pour un même couple (clé, nonce), ce qui suppose une collision de sel, de probabilité d'environ $n^2/2^{129}$, soit $1{,}5 \cdot 10^{-21}$ pour $n = 10^9$ fichiers. (Avec une clé *fixe*, des nonces aléatoires de 96 bits limiteraient à environ 2^{32} messages.)

3. Et si le nom n'était pas dans les données associées ? Le chiffré ne serait lié à aucun contexte. Un attaquant qui peut écrire dans le dossier copie le contenu de `public.txt.cfr` dans `virement.txt.cfr` : si les deux sont chiffrés avec le même mot de passe, le déchiffrement réussit et rend un contenu choisi par l'attaquant (substitution). Avec le nom dans les données associées, on obtient `InvalidTag` (test du fichier renommé).

4. Pourquoi authentifier l'en-tête ? Pour garantir que la version, les paramètres, le sel et le nonce lus sont ceux du chiffrement et éviter les rétrogradations ou ambiguïtés entre versions. Modifier (t, m, p) ou le sel change aussi la clé dérivée, donc l'étiquette échoue déjà : c'est une défense en profondeur (le test de l'en-tête modifié le montre). Limite : les paramètres étant lus avant l'authentification, un attaquant peut les gonfler pour provoquer un déni de service, d'où les bornes imposées.

5. Pourquoi masquer la raison précise d'un échec ? Des messages distincts (mauvais mot de passe, fichier modifié, mauvais nom, paramètres invalides) donneraient à l'attaquant un oracle lui indiquant quelle partie de sa tentative est valide. Un message unique et neutre ne révèle rien ; le détail va dans un journal interne.

7 Bilan du TP 1

Le module respecte le format demandé et résiste aux cinq attaques testées (mauvais mot de passe, altération du chiffré, de l'en-tête, renommage, format inconnu). Les paramètres Argon2id sont réglés à environ 0,5 s. Améliorations possibles : migration paresseuse des paramètres à l'ouverture, écriture atomique lors du rechiffrement, gestion mémoire des secrets.

Deuxième partie

TP 2 – Construire une infrastructure de gestion de clés

1 Objectifs et architecture

On construit avec OpenSSL une PKI à deux niveaux, on émet des certificats de serveur et de client, on valide les chaînes et on révoque un certificat.

Élément	Clé et durée	Extensions essentielles
AC racine	P-384, 10 ans, hors ligne	`CA:TRUE, pathlen:1` ; `keyCertSign`, `cRLSign`
AC intermédiaire	P-256, 5 ans	`CA:TRUE, pathlen:0` ; contraintes de nom `tp.local`
Serveur `www.tp.local`	P-256, 90 jours	`CA:FALSE` ; `serverAuth` ; noms alternatifs
Client `alice@tp.local`	P-256, 90 jours	`CA:FALSE` ; `clientAuth`

Chaque autorité possède sa base : `index.txt` (certificats émis ou révoqués), `serial`, `crlnumber` et le dossier `newcerts/`.

2 Étape 1 – Les autorités et leurs fichiers de configuration

Les fichiers `racine.cnf` et `inter.cnf` sont en annexe B.

Ligne	Rôle
`basicConstraints = critical,CA:TRUE,pathlen:0`	ce certificat est une AC qui ne peut signer que des certificats finaux ; `critical` : un client qui ne comprend pas l'extension doit rejeter le certificat
`keyUsage = critical,keyCertSign,cRLSign`	la clé ne sert qu'à signer des certificats et des CRL
`subjectKeyIdentifier` et `authorityKeyIdentifier`	empreinte de la clé du sujet et de la clé de l'émetteur : chaînage et choix de la bonne clé
`nameConstraints = critical,permitted;DNS:tp.local,...`	l'intermédiaire ne peut émettre que pour `tp.local`, ses sous-domaines et les adresses `@tp.local` ; vérifié par les clients
`extendedKeyUsage = serverAuth` / `clientAuth`	restreint l'usage du certificat final
`basicConstraints = critical,CA:FALSE`	un certificat final ne peut pas signer
`policy` (`match`, `supplied`)	champs du sujet qui doivent correspondre à ceux de l'AC, ou être fournis
`copy_extensions`	`none` pour la racine (elle décide seule) ; `copy` pour l'intermédiaire (reprise du SAN demandé)
`unique_subject = no`	autorise la réémission pour le même sujet (renouvellement, TP 3)
`default_crl_days`	validité d'une CRL : 30 jours (racine), 7 jours (intermédiaire)

Attention à `copy_extensions = copy` : les extensions de la CSR sont reprises si la section de l'AC ne les définit pas ; ici `ext_serveur` et `ext_client` définissent `basicConstraints`, qui l'emporte. En production, l'opérateur relit toujours la CSR avant de signer.

3 Étape 2 – AC racine

Clé P-384 (`openssl genpkey`), puis certificat auto-signé (`openssl req -x509 -addext ...`), commandes reprises dans `pki.sh`. Lecture de `openssl x509 -text` :

Champ	Valeur relevée
Émetteur et sujet	`C = BF, O = Universite TP, CN = Racine TP` (identiques)
Validité	du 24 octobre 2026 au 21 octobre 2036 (10 ans)
Clé publique	`id-ecPublicKey`, courbe `secp384r1` (P-384)
Signature	`ecdsa-with-SHA384`
Extensions critiques	`Basic Constraints: CA:TRUE, pathlen:1` ; `Key Usage: Certificate Sign, CRL Sign`

Pourquoi l'émetteur et le sujet sont-ils identiques ? La racine est auto-signée : elle n'a pas d'autorité supérieure. Sa confiance ne vient pas d'une signature (n'importe qui peut fabriquer un certificat auto-signé) mais de son installation préalable, hors bande, dans le magasin de confiance des clients (ancre de confiance). `pathlen:1` autorise une seule AC intermédiaire sous la racine.

4 Étape 3 – AC intermédiaire

Génération de la clé P-256 et de la CSR (sujet `/C=BF/O=Universite TP/CN=Intermediaire TP`), puis signature par la racine : `openssl ca -batch -notext -config racine.cnf -extensions ext_inter -days 1825`. Contrôle des extensions dans le certificat obtenu :

```
$ openssl x509 -in inter/inter.crt -noout -ext basicConstraints,nameConstraints
X509v3 Basic Constraints: critical
    CA:TRUE, pathlen:0
X509v3 Name Constraints: critical
    Permitted:
      DNS:tp.local
      email:tp.local
```

Le certificat porte le numéro de série 1000 et est valable du 24 octobre 2026 au 23 octobre 2031 (5 ans). Les extensions proviennent de la configuration de la racine, non de la CSR (`copy_extensions = none`). La politique de la racine impose `match` sur le pays et l'organisation : un sujet différent ferait échouer l'émission.

5 Étape 4 – Certificats de serveur et de client

Chaque demandeur génère sa propre clé P-256 et sa CSR (avec les noms alternatifs) ; l'intermédiaire signe avec `ext_serveur` ou `ext_client`, pour 90 jours.

Certificat	Série	Validité	Usage étendu	SAN
`serveur.crt`	1000	24/10/26 – 22/01/27	`serverAuth`	`DNS:www.tp.local, DNS:tp.local`
`alice.crt`	1001	24/10/26 – 22/01/27	`clientAuth`	`email:alice@tp.local`
`bob.crt`	1002	24/10/26 – 22/01/27	`clientAuth`	`email:bob@tp.local`

Le certificat de Bob est émis dès maintenant car celui d'Alice sera révoqué et le TP 3 a besoin d'un client valide.

6 Étape 5 – Validation

Commande (résumé)	Attendu	Observé
`verify -verify_hostname www.tp.local serveur.crt`	OK	`serveur.crt: OK`
`verify -verify_hostname autre.example serveur.crt`	échec	`error 62 ... hostname mismatch`
`verify -purpose sslclient alice.crt`	OK	`alice.crt: OK`
`verify -purpose sslserver alice.crt` (contre-épreuve)	échec	`error 26 ... unsuitable certificate purpose`
`verify` d'un certificat pour `www.example.com`	échec	`error 47 ... permitted subtree violation`

Interprétation. `-CAfile` désigne les ancres de confiance (la racine seule) ; `-untrusted` fournit les intermédiaires nécessaires à la construction du chemin sans leur accorder de confiance. La validation contrôle le chemin de certification, la signature de chaque certificat, la validité temporelle, `basicConstraints` et `pathlen`, `keyUsage`/`extendedKeyUsage`, les contraintes de nom, le nom d'hôte (SAN) et, si on le demande, la révocation. Le *hostname mismatch* montre que la chaîne seule ne suffit pas : le client doit aussi vérifier que le certificat est valable pour le nom visé.

Pourquoi l'AC a-t-elle pu signer `www.example.com` ? La commande `openssl ca` a répondu `Database updated` : elle applique seulement la politique de `inter.cnf`, qui ne limite pas les noms, et ne relit pas les contraintes de nom inscrites par la racine dans le certificat de l'intermédiaire. Ce sont les clients qui les appliquent à la validation. C'est un garde-fou externe qui limite les dégâts en cas d'abus ou de vol de la clé de l'intermédiaire (impossible d'émettre pour un autre domaine), mais il ne dispense pas l'AC de vérifier les demandes.

7 Étape 6 – Révocation

Hypothèse : la clé d'Alice est compromise. On révoque son certificat (`-revoke alice.crt -crl_reason keyCompromise`), on génère les CRL de l'intermédiaire et de la racine et on les concatène dans `crls.pem`. Dans `inter/index.txt`, la ligne d'Alice passe de `V` à `R`, avec la date et la raison (`keyCompromise`).

Vérification avec `-crl_check_all`	Résultat
`alice.crt` (révoqué)	`error 23 ... certificate revoked`
`bob.crt` (non révoqué)	`bob.crt: OK`
`bob.crt` sans fournir de CRL	`error 3 ... unable to get certificate CRL`

Champ de la CRL de l'intermédiaire	Valeur
Émetteur	`C = BF, O = Universite TP, CN = Intermediaire TP`
Last Update / Next Update	24 octobre 2026 / 31 octobre 2026 (7 jours)
Certificat révoqué	série 1001, révoqué le 24 octobre 2026, raison *Key Compromise*
Signature	`ecdsa-with-SHA256`

La CRL ne contient ni le nom ni la clé du certificat révoqué, seulement son numéro de série. L'option `-crl_check_all` contrôle toute la chaîne : il faut donc une CRL pour l'intermédiaire *et* une pour la racine (vide mais présente) ; sans CRL, la vérification échoue (échec fermé). Une CRL a une date d'expiration : elle doit être régénérée et republiée avant `nextUpdate`.

8 Étape 7 – Script `pki.sh`

Le script (annexe C) regroupe les étapes 2 à 4, plus l'émission de Bob et la génération des CRL initiales. Il commence par `set -euo pipefail` : arrêt à la première erreur (`-e`), variable non

définie traitée comme une erreur (`-u`), propagation d'un échec à l'intérieur d'un tube (`pipefail`). Il refuse de s'exécuter si une racine existe déjà, utilise `umask 077` pour que les clés privées ne soient lisibles que par leur propriétaire, et factorise l'émission des certificats feuilles dans une fonction `emettre`. Exécuté sur un dossier vide, il a produit une PKI dont les certificats passent toutes les validations de l'étape 5.

9 Questions de réflexion

1. Racine hors ligne ; compromission de l'intermédiaire ou de la racine. La racine est l'ancre de confiance de tous les clients : si sa clé fuit, un attaquant émet des certificats acceptés partout. Elle ne sert que rarement (signer l'intermédiaire, publier sa CRL), donc on la garde hors ligne (machine déconnectée, HSM). Si l'*intermédiaire* est compromis : la racine le révoque, on génère une nouvelle clé, on la fait signer et on réémet les certificats feuilles ; `pathlen:0` et les contraintes de nom ont limité les dégâts. Si la *racine* est compromise : la révocation est impossible (elle signe sa propre CRL), il faut créer une nouvelle racine, la redistribuer comme ancre de confiance à tous les clients et tout réémettre.

2. Rôle de `pathlen:0`. L'intermédiaire ne peut signer que des certificats finaux. Sans cette limite, un abus ou le vol de sa clé permettrait de créer des AC subordonnées non prévues, émettant des certificats valides en dehors de tout contrôle et de tout inventaire.

3. Pourquoi la clé du serveur est-elle générée par le serveur ? La clé privée ne doit jamais quitter son propriétaire : l'AC n'a besoin que de la clé publique, transmise dans la CSR. La CSR est signée avec la clé privée, ce qui prouve la possession de la clé. Générée par l'AC, la clé devrait être transportée (risque d'interception), pourrait être copiée (usurpation) et la non-répudiation serait perdue.

4. CN ou SAN ? Les clients modernes vérifient le **SAN** ; le CN est déprécié et ignoré dès qu'un SAN de type DNS existe. On conserve le CN pour l'affichage et les vieux clients, mais tous les noms doivent figurer dans le SAN.

5. Limites des CRL ; comparaison avec OCSP et les certificats de courte durée.

	CRL	OCSP	Courte durée
Principe	liste signée, téléchargée	question en ligne sur un numéro de série	le certificat expire seul
Fraîcheur	limitée par `nextUpdate`	quasi temps réel	délai $\leq$ durée de vie
Coût	liste qui grossit, téléchargement	une requête par connexion	renouvellement très fréquent
Vie privée	bonne	le répondeur voit les sites visités (agrafage OCSP)	bonne
Faiblesse	distribution, clients laxistes	disponibilité, *soft-fail*	exige l'automatisation (ACME)

La tendance est d'automatiser et de raccourcir les durées, ce qui rend la révocation presque sans objet.

6. Politique de certification minimale.

1. **Périmètre** : certificats pour le domaine `tp.local` (serveurs) et les adresses `@tp.local` (clients).
2. **Demandeurs** : un administrateur désigné par domaine ; chaque membre pour son propre certificat client.

3. **Identité** : maîtrise du nom prouvée pour un serveur ; compte institutionnel et pièce d'identité pour une personne.
4. **Clés** : générées par le demandeur (EC P-256 minimum), jamais transmises ; clé de la racine hors ligne, protégée par phrase secrète.
5. **Demandes** : CSR obligatoire, relue par un opérateur avant signature.
6. **Durées** : feuilles 90 jours, intermédiaire 5 ans, racine 10 ans.
7. **Extensions** : usage et `extendedKeyUsage` imposés par l'AC ; `CA:FALSE` pour les feuilles.
8. **Révocation** : sur demande du titulaire ou de l'AC, traitée sous 24 h ; CRL republiée à chaque révocation et au moins tous les 7 jours.
9. **Journalisation** : toute émission et révocation consignée dans `index.txt` sauvegardé ; audit annuel.
10. **Compromission** : procédure documentée (révocation, réémission, information des utilisateurs).

10 Bilan du TP 2

La PKI est opérationnelle : chaîne racine–intermédiaire–feuilles valide, contraintes de nom et `pathlen` effectifs, révocation observée par la validation. Principales erreurs à éviter : utiliser le mauvais fichier de configuration pour signer, oublier l'une des deux CRL avec `-crl_check_all`, laisser expirer une CRL.

Troisième partie

TP 3 – Déployer un canal sécurisé et gérer les certificats

1 Objectifs et points d'attention

On déploie nginx en TLS 1.2/1.3 avec authentification mutuelle (mTLS), on analyse la poignée de main, on vérifie la configuration, on supervise les expirations et on automatise le renouvellement, avec la PKI du TP 2. Trois points de l'énoncé ont nécessité une adaptation, décrits aux étapes concernées : la *chaîne du client*, la *liste des suites de chiffrement* et le *test de TLS 1.1*.

2 Étape 1 – Préparation des fichiers

```
cat serveur.crt inter/inter.crt > serveur_chaine.crt
sudo install -m 600 serveur.key /etc/nginx/tls/serveur.key
sudo install -m 644 serveur_chaine.crt racine/racine.crt crls.pem /etc/nginx/tls/
echo "127.0.0.1 www.tp.local" | sudo tee -a /etc/hosts
cat bob.crt inter/inter.crt > bob_chaine.crt
```

Le serveur présente son certificat suivi de l'intermédiaire, jamais la racine. La clé est en droits `600`, les fichiers publics en `644`. Le nom de laboratoire est associé à la machine locale pour que le SAN corresponde.

3 Étape 2 – Configuration de nginx

La configuration finale (celle de l'énoncé, durcie à l'étape 4) est en annexe D. `nginx -t` répond `test is successful` ; le rechargement est sans coupure.

Directive	Justification
`ssl_certificate`	certificat du serveur suivi de l'intermédiaire
`ssl_protocols TLSv1.2 TLSv1.3`	exclut SSL 3 et TLS 1.0/1.1, obsolètes (RFC 8996) et vulnérables (POODLE, BEAST)
`ssl_prefer_server_ciphers off`	le client choisit la meilleure suite pour son matériel (AES accéléré ou ChaCha20), tous les choix proposés étant sûrs
`ssl_ciphers`, `ssl_ecdh_curve`	**ajoutées** à l'étape 4 : ECDHE et AEAD uniquement ; groupes X25519, P-256, P-384
`ssl_session_tickets off`	évite qu'une clé de ticket de longue durée compromette la confidentialité persistante (question 2)
`ssl_client_certificate`	AC de confiance pour les clients
`ssl_verify_client on`	certificat client obligatoire et vérifié
`ssl_verify_depth 2`	nombre maximal d'intermédiaires acceptés : essai de profondeur 1 réussi, profondeur 0 refusée ; 2 laisse une marge
`ssl_crl`	révocation : une CRL par niveau de chaîne ; sans CRL de l'émetteur, refus ; une CRL expirée ferait refuser tous les clients
`add_header Strict-Transport-Security ... always`	HSTS, y compris dans les réponses d'erreur
`return 200 "Bonjour $ssl_client_s_dn"`	renvoie le DN du client authentifié, preuve que le mTLS fonctionne

4 Étape 3 – Observation de la poignée de main

4.1 Connexions testées

Cas	Résultat observé	Explication
Bob avec sa chaîne (`bob_chaine.crt`)	`Bonjour CN=bob@tp.local`	certificat valide, chaîne complète
Bob avec son seul certificat	`400 The SSL certificate error`	nginx ne peut pas construire le chemin sans l'intermédiaire
Aucun certificat client	`400 No required SSL certificate was sent`	journal : *client sent no required SSL certificate*
Alice (révoquée)	`400 The SSL certificate error`	journal : *client SSL certificate verify error : (23 :certificate revoked)*

Adaptation 1 – chaîne du client. L'énoncé fournit à curl le seul `bob.crt`, ce qui échoue car `ssl_client_certificate` ne contient que la racine. Deux solutions ont été testées avec succès : le client envoie sa chaîne (`bob_chaine.crt`), ou l'on ajoute l'intermédiaire au fichier des AC de confiance du serveur. Observation : nginx ne coupe pas la connexion par une alerte TLS ; il termine la poignée de main puis répond HTTP 400, et le motif exact est dans le journal d'erreurs (niveau `info`).

4.2 Paramètres de la connexion TLS

```
$ openssl s_client -connect www.tp.local:443 -servername www.tp.local \
    -verify_hostname www.tp.local -CAfile racine/racine.crt \
    -cert bob.crt -key bob.key -cert_chain inter/inter.crt -brief
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN = www.tp.local
Verification: OK
Verified peername: www.tp.local
Server Temp Key: X25519, 253 bits
```

Paramètre	Valeur et commentaire
Version	TLS 1.3 (la meilleure version commune) ; en forçant `-tls1_2` : TLS 1.2
Suite	`TLS_AES_256_GCM_SHA384` (AEAD) ; en TLS 1.2 : `ECDHE-ECDSA-AES256-GCM-SHA384`
Échange de clés	ECDHE sur **X25519** : clé éphémère, donc confidentialité persistante
Vérification	`Verification: OK` : chaîne valide jusqu'à la racine et nom conforme au SAN

4.3 Analyse des messages de la poignée de main

La capture sur l'interface de bouclage n'étant pas possible dans notre environnement conteneurisé, l'analyse a été faite avec `openssl s_client -trace`, qui détaille chaque enregistrement TLS et son type externe (ce que Wireshark montrerait avec les filtres `tls.handshake.type == 1` et `== 2`). Séquence des messages *reçus par le client* :

TLS 1.3	TLS 1.2
ServerHello (record *Handshake*, **en clair**)	ServerHello (en clair)
ChangeCipherSpec (compatibilité)	**Certificate** (1035 octets, **en clair**)
record *ApplicationData* → EncryptedExtensions	ServerKeyExchange (en clair)
record *ApplicationData* → CertificateRequest	CertificateRequest (en clair)
record *ApplicationData* → **Certificate** (1040 octets)	ServerHelloDone
record *ApplicationData* → CertificateVerify, Finished	... (le certificat du client aussi est en clair)

ClientHello (en clair) : extension `server_name` (le nom `www.tp.local` est visible : SNI), `supported_versions`, `key_share` avec le groupe `x25519`. **ServerHello** : suite retenue `0x1302` (`TLS_AES_256_GCM_SHA384`), `supported_versions` (TLS 1.3) et `key_share` du serveur.

Pourquoi le certificat du serveur n'est-il pas visible en TLS 1.3 ? Dès le ServerHello, les deux parties disposent du secret ECDHE partagé et dérivent des clés de poignée de main : tous les messages suivants, dont *Certificate*, voyageaient chiffrés, avec le type externe *ApplicationData* (seul le client, qui déchiffre, voit le type interne *Handshake*). En TLS 1.2, le message *Certificate* circulait en clair. En TLS 1.3, l'identité du serveur (hors SNI) et celle du client sont donc protégées contre un observateur. Pour déchiffrer sa propre capture avec Wireshark : `SSLKEYLOGFILE=~/cles.log curl ...`, puis indiquer ce fichier dans les préférences TLS.

5 Étape 4 – Vérification de la configuration

TLS 1.1 doit être refusé.

```
$ openssl s_client -connect www.tp.local:443 -tls1_1 -brief
no protocols available            # refus CLIENT (Ubuntu impose TLS >= 1.2) : ne prouve rien
$ OPENSSL_CONF=/dev/null openssl s_client -connect www.tp.local:443 \
    -tls1_1 -cipher 'DEFAULT:@SECLEVEL=0' -brief
tlsv1 alert protocol version      # alerte 70 envoyee par le SERVEUR : preuve du refus
```

Adaptation 3. Le test de l'énoncé échoue côté client à cause de la configuration d'OpenSSL ; il faut la neutraliser pour que le refus vienne bien du serveur.

Suites acceptées en TLS 1.2. Avec la configuration initiale, `nmap -script ssl-enum-ciphers -p 443 www.tp.local` liste 14 suites, dont des suites **CBC**, `CCM_8`, `ARIA` et `CAMELLIA`. Elles offrent la confidentialité persistante (ECDHE) mais, pour CBC, pas de chiffrement authentifié (MAC puis chiffrement, surface d'attaques de type Lucky13). **Adaptation 2** : ajout de

```
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-
    POLY1305;
ssl_ecdh_curve X25519:prime256v1:secp384r1;
```

Après rechargement, nmap ne liste plus que trois suites en TLS 1.2 (`ECDHE_ECDSA` avec AES-128-GCM, AES-256-GCM et ChaCha20-Poly1305) et les trois suites de TLS 1.3, toutes de note A. Ces tests n'ont été réalisés que sur le serveur de laboratoire local.

6 Étape 5 – Supervision : `surveiller.py`

Le script (annexe E) lit tous les certificats d'un dossier (un fichier de chaîne en contient plusieurs), calcule les jours restants et affiche un niveau : OK, ALERTE (moins de 30 jours), CRITIQUE (moins de 7 jours). Le code de sortie vaut le pire niveau (0, 1 ou 2), convention de Nagios et de la plupart des outils. Les fichiers de clé ou de CRL sont ignorés ; un certificat déjà expiré est CRITIQUE.

```
$ python3 surveiller.py /tmp/certs ; echo "exit=$?"
OK           89 j  bob.crt  [CN=bob@tp.local]
CRITIQUE      4 j  court.crt  [CN=court.tp.local]
OK         1824 j  inter.crt  [CN=Intermediaire TP,O=Universite TP,C=BF]
ALERTE       19 j  vingt.crt  [CN=court.tp.local]
OK           89 j  serveur.crt  [CN=www.tp.local]
exit=2
```

Certificat testé	Jours restants	Niveau	Code de sortie
`serveur.crt`	89	OK	0
certificat de 20 jours	19	ALERTE	1
certificat de 5 jours	4	CRITIQUE	2

Un certificat émis pour 5 jours affiche 4 jours : le calcul tronque (il reste 4 jours et quelques heures), choix prudent pour une supervision. Intégration possible : `surveiller.py /etc/nginx/tls || envoyer_alerte`.

7 Étape 6 – Renouvellement : `renouveler.sh`

Le script (annexe F) : (1) ne fait rien si le certificat reste valable plus de 30 jours (`openssl x509 -checkend`) ; (2) génère une *nouvelle* clé et une demande ; (3) la fait signer par l'intermédiaire ; (4) vérifie la chaîne, l'usage serveur, le nom et la correspondance clé/certificat *avant* tout déploiement ; (5) sauvegarde l'ancienne clé et l'ancienne chaîne dans `archive/` horodaté ; (6) installe, teste `nginx -t`, recharge sans coupure ; (7) contrôle que le serveur présente bien le nouveau certificat (comparaison d'empreintes) et revient en arrière sinon.

```
$ ./renouveler.sh                          # certificat valable plus de 30 jours
renouveler: certificat valide plus de 30 jours : rien a faire          (exit 0)
$ CN=www.example.com ... FORCE=1 ./renouveler.sh   # nom hors perimetre
error 47 at 0 depth lookup: permitted subtree violation               (rien deploye)
$ SEUIL=100 ./renouveler.sh                # seuil releve pour forcer le test
renouveler: nouvelle chaine verifiee
renouveler: OK : nouveau certificat en service (notAfter=Jan 22 ... 2027 GMT)
                                           # serie 1000 -> 1007 ; archive/ cree ; mTLS toujours
    OK
$ CIBLE=127.0.0.1:9 SEUIL=100 ./renouveler.sh      # controle final en echec simule
renouveler: ECHEC : retour a la version precedente                      (exit 1)
                                           # numero de serie inchange, le site repond toujours
```

Défaut trouvé et corrigé pendant la mise au point : avec `set -e`, l'échec du contrôle final terminait le script sans déclencher le retour arrière ; la commande est maintenant suivie de `|| SERVI=""` puis d'une comparaison explicite. Le test d'échec simulé confirme le retour arrière.

Planification (minuteur systemd ou cron, exécution quotidienne ; le script ne renouvelle que si l'expiration est dans moins de 30 jours) :

```
# /etc/systemd/system/renouveler-tls.service
[Unit]
Description=Renouvellement du certificat TLS du serveur
[Service]
Type=oneshot
Environment=PKI=/opt/pki
ExecStart=/usr/local/sbin/renouveler.sh

# /etc/systemd/system/renouveler-tls.timer
[Unit]
Description=Execution quotidienne du renouvellement TLS
[Timer]
```

```
OnCalendar=daily
RandomizedDelaySec=1h
Persistent=true
[Install]
WantedBy=timers.target

# activation :  sudo systemctl enable --now renouveler-tls.timer
# variante cron (/etc/cron.d/renouveler-tls) :
17 3 * * * root PKI=/opt/pki /usr/local/sbin/renouveler.sh >> /var/log/renouveler.log 2>&1
```

Réserve : dans le TP, le serveur détient la clé de l'intermédiaire (`openssl ca` local), ce qui est inacceptable en production ; on utiliserait un protocole automatisé (ACME : `certbot`, `acme.sh`) auprès d'une AC dont la clé reste protégée. La structure du script (demande, vérification, déploiement, retour arrière) reste la même.

8 Questions de réflexion

1. Pourquoi ne pas envoyer la racine ? Le client la possède déjà dans son magasin de confiance ; l'envoyer coûte de la bande passante à chaque connexion et n'apporte aucune sécurité : un client ne fait jamais confiance à une racine parce qu'un serveur la lui présente.

2. Intérêt de désactiver les tickets de session. Un ticket contient l'état de la session chiffré par une clé de ticket du serveur. Si cette clé n'est pas renouvelée (nginx la garde jusqu'au rechargement), qui la vole peut déchiffrer rétroactivement les sessions reprises : la confidentialité persistante d'ECDHE est perdue. Désactiver les tickets impose une poignée de main complète (un peu plus de calcul) mais préserve cette propriété.

3. Que garantit HSTS ? Le navigateur n'utilise plus que HTTPS pour le site pendant `max-age` secondes (31 536 000 s, un an) et ne permet pas de passer outre une erreur de certificat. Il protège contre le *SSL stripping* (réécriture des liens https en http par un intermédiaire) et contre les rétrogradations. Limites : la première visite reste exposée (liste de préchargement) ; une erreur de configuration peut verrouiller les utilisateurs pour un an, d'où des essais avec une durée courte.

4. Certificats de 100 jours (2027) puis 47 jours (2029). Pour les certificats publics, le renouvellement devient très fréquent (une dizaine de fois par an et par certificat), avec une validation de domaine réutilisable de moins en moins longtemps. Une organisation qui procède à la main aura des oublis, donc des pannes par expiration, une charge ingérable et pas d'inventaire. Il faut automatiser (ACME), inventorier, superviser (étape 5) et tester le renouvellement, et prévoir la crypto-agilité. (Cela concerne les certificats publics, non la PKI interne du laboratoire.)

5. Chaîne incomplète : pourquoi certains clients l'acceptent-ils ? Les navigateurs peuvent retrouver l'intermédiaire manquant (cache, extension *Authority Information Access*, préchargement). Les bibliothèques comme OpenSSL, curl, Java ou Python ne le font pas et rejettent la connexion (`unable to get local issuer certificate`) : le site « marche dans mon navigateur » mais pas pour les robots. D'où la nécessité de servir la chaîne complète, sauf la racine.

6. Procédure de réaction si la clé privée du serveur fuit.

1. **Contenir** : isoler la machine, identifier le périmètre et la cause (accès, sauvegarde, dépôt de code).
2. **Révoquer** le certificat avec la raison `keyCompromise`, régénérer et republier les CRL, puis recharger les services qui les utilisent (nginx relit `ssl_crl` au rechargement).
3. **Nouvelle clé et nouveau certificat** (`FORCE=1 ./renouveler.sh`) : jamais de réutilisation de l'ancienne clé ; déployer et contrôler.

4. **Invalider les sessions** : redémarrer le service (clés de tickets) ; changer les secrets qui ont transité par ce serveur.
5. **Évaluer l'impact** : grâce à ECDHE, le trafic passé n'est pas déchiffrable avec la seule clé du certificat, mais le serveur a pu être usurpé jusqu'à la révocation.
6. **Informer**, consigner dans un rapport d'incident et corriger la cause (permissions, gestion des secrets, analyse des dépôts).

9 Bilan du TP 3

Le serveur impose TLS 1.2/1.3 avec uniquement des suites ECDHE et AEAD, authentifie les clients par certificat, refuse le certificat révoqué d'Alice et présente la chaîne sans la racine. La supervision signale les expirations avec un code de sortie exploitable, et le renouvellement est automatisé avec vérification préalable et retour arrière testé. Enseignements : la chaîne côté client, l'expiration des CRL et le test réel du refus des anciens protocoles sont les points qui piègent le plus souvent.

Conclusion générale

Les trois TP forment une chaîne cohérente : un chiffrement authentifié correctement assemblé (TP 1), une infrastructure de gestion de clés avec contraintes et révocation (TP 2), puis son exploitation dans un canal TLS avec authentification mutuelle, supervision et renouvellement automatique (TP 3). Dans chaque cas, les propriétés de sécurité ont été *démontrées par des tests ou des observations* (exceptions `InvalidTag`, erreurs de validation, refus du serveur, retour arrière) et non supposées. Le fil conducteur est de ne jamais réinventer une primitive, de borner ce qui est lu avant authentification, d'automatiser tout ce qui expire et de vérifier avant de déployer.

Annexes

A TP 1 – Code source

A.1 `coffre.py`

```python
"""coffre.py - mini-coffre de fichiers (format CFR1)."""
import os
import struct

from argon2.low_level import hash_secret_raw, Type
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

MAGIC = b"CFR1"
T, M, P = 3, 128 * 1024, 4          # temps, memoire (Kio), parallelisme
LEN_SEL, LEN_NONCE, LEN_TAG = 16, 12, 16
LEN_ENTETE = 4 + 12 + LEN_SEL + LEN_NONCE   # 44 octets
M_MAX, T_MAX, P_MAX = 1024 * 1024, 20, 16    # bornes anti-DoS (1 Gio)


def cle_maitresse(mot_de_passe: str, sel: bytes, t=T, m=M, p=P) -> bytes:
    """Argon2id -> 32 octets."""
    return hash_secret_raw(secret=mot_de_passe.encode("utf-8"), salt=sel,
                           time_cost=t, memory_cost=m, parallelism=p,
                           hash_len=32, type=Type.ID)


def sous_cle(maitresse: bytes, usage: bytes) -> bytes:
    """HKDF-SHA256 avec info = b"coffre-v1|" + usage -> 32 octets."""
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
                info=b"coffre-v1|" + usage).derive(maitresse)


def _cle_fichier(mdp, sel, t, m, p) -> bytes:
    return sous_cle(cle_maitresse(mdp, sel, t, m, p), b"chiffrement-fichier")


def chiffrer(mot_de_passe: str, nom: str, donnees: bytes,
             t=T, m=M, p=P) -> bytes:
    sel, nonce = os.urandom(LEN_SEL), os.urandom(LEN_NONCE)
    entete = MAGIC + struct.pack(">III", t, m, p) + sel + nonce
    cle = _cle_fichier(mot_de_passe, sel, t, m, p)
    ad = entete + nom.encode("utf-8")
    return entete + AESGCM(cle).encrypt(nonce, donnees, ad)


def dechiffrer(mot_de_passe: str, nom: str, blob: bytes) -> bytes:
    if len(blob) < LEN_ENTETE + LEN_TAG or blob[:4] != MAGIC:
        raise ValueError("format inconnu")
    t, m, p = struct.unpack(">III", blob[4:16])
    if not (1 <= t <= T_MAX and 8 * p <= m <= M_MAX and 1 <= p <= P_MAX):
        raise ValueError("parametres hors bornes")
    sel = blob[16:16 + LEN_SEL]
    nonce = blob[32:32 + LEN_NONCE]
    entete = blob[:LEN_ENTETE]
    cle = _cle_fichier(mot_de_passe, sel, t, m, p)
    ad = entete + nom.encode("utf-8")
    return AESGCM(cle).decrypt(nonce, blob[LEN_ENTETE:], ad)  # InvalidTag


def rechiffrer(ancien_mdp: str, nouveau_mdp: str, nom: str,
```

```
                blob: bytes, t=T, m=M, p=P) -> bytes:
    """Change le mot de passe et applique les parametres courants."""
    donnees = dechiffrer(ancien_mdp, nom, blob)
    return chiffrer(nouveau_mdp, nom, donnees, t, m, p)
```

A.2 test_coffre.py

```
import pytest
from cryptography.exceptions import InvalidTag
import coffre

FAST = dict(t=1, m=8 * 1024, p=4)        # parametres legers pour les tests
MDP, NOM, DATA = "correct horse battery", "rapport.txt", b"donnees secretes" * 10


def chif(mdp=MDP, nom=NOM, data=DATA):
    return coffre.chiffrer(mdp, nom, data, **FAST)


def test_aller_retour():
    assert coffre.dechiffrer(MDP, NOM, chif()) == DATA


def test_chiffrement_probabiliste():
    a, b = chif(), chif()
    assert a != b and a[16:44] != b[16:44]       # sel et nonce differents


def test_mauvais_mot_de_passe():
    with pytest.raises(InvalidTag):
        coffre.dechiffrer("mauvais", NOM, chif())


@pytest.mark.parametrize("pos", [coffre.LEN_ENTETE + 3, -1])
def test_octet_modifie(pos):
    blob = bytearray(chif())
    blob[pos] ^= 0x01
    with pytest.raises(InvalidTag):
        coffre.dechiffrer(MDP, NOM, bytes(blob))


def test_entete_modifie():
    blob = bytearray(chif())
    blob[20] ^= 0x01                             # un octet du sel
    with pytest.raises(InvalidTag):
        coffre.dechiffrer(MDP, NOM, bytes(blob))


def test_fichier_renomme():
    with pytest.raises(InvalidTag):
        coffre.dechiffrer(MDP, "autre.txt", chif())


def test_format_inconnu():
    blob = b"XXXX" + chif()[4:]
    with pytest.raises(ValueError):
        coffre.dechiffrer(MDP, NOM, blob)


def test_rechiffrement():
    blob = coffre.rechiffrer(MDP, "nouveau", NOM, chif(), **FAST)
    assert coffre.dechiffrer("nouveau", NOM, blob) == DATA
    with pytest.raises(InvalidTag):
```

```
        coffre.dechiffrer(MDP, NOM, blob)


def test_anciens_parametres_lisibles():
    ancien = coffre.chiffrer(MDP, NOM, DATA, t=1, m=8 * 1024, p=4)
    nouveau = coffre.chiffrer(MDP, NOM, DATA, t=2, m=16 * 1024, p=4)
    assert coffre.dechiffrer(MDP, NOM, ancien) == coffre.dechiffrer(MDP, NOM, nouveau)
```

A.3 demo.py

```
"""demo.py - demonstration du coffre (TP 1)."""
import coffre

MDP, NOM = "motdepasse-demo", "note.txt"
data = b"Contenu confidentiel du fichier.\n" * 30

b = coffre.chiffrer(MDP, NOM, data)
print(f"clair : {len(data)} octets ; chiffre : {len(b)} octets "
      f"(= {len(data)} + 44 + 16)")
print("en-tete (hex) :", b[:4].hex(), "|", b[4:16].hex(), "|",
      b[16:32].hex(), "|", b[32:44].hex())
t, m, p = (int.from_bytes(b[i:i + 4], "big") for i in (4, 8, 12))
print(f"identifiant = {b[:4]!r} ; t, m, p = {t}, {m} Kio ({m // 1024} Mio), {p}")
b2 = coffre.chiffrer(MDP, NOM, data)
print("deux chiffrements identiques ?", b == b2, "; sels egaux ?", b[16:32] == b2[16:32])
print("dechiffrement correct ?", coffre.dechiffrer(MDP, NOM, b) == data)
n = coffre.rechiffrer(MDP, "nouveau-mdp", NOM, b, t=4, m=128 * 1024, p=4)
print("apres rechiffrement : t =", int.from_bytes(n[4:8], "big"))
try:
    coffre.dechiffrer(MDP, NOM, n)
except Exception as e:
    print("ancien mot de passe ->", type(e).__name__)
print("nouveau mot de passe correct ?", coffre.dechiffrer("nouveau-mdp", NOM, n) == data)
```

B TP 2 – Fichiers de configuration

B.1 racine.cnf

```
# racine.cnf - AC racine (hors ligne) : signe l'intermediaire et sa propre CRL
[ ca ]
default_ca = CA_racine

[ CA_racine ]
dir               = ./racine
database          = $dir/index.txt
new_certs_dir     = $dir/newcerts
serial            = $dir/serial
crlnumber         = $dir/crlnumber
certificate       = $dir/racine.crt
private_key       = $dir/racine.key
default_md        = sha384
default_days      = 1825
default_crl_days  = 30
unique_subject    = no
policy            = policy_racine
copy_extensions   = none
crl_extensions    = ext_crl

[ policy_racine ]
countryName       = match
```

```
organizationName = match
commonName       = supplied

# Extensions imposees a l'AC intermediaire
[ ext_inter ]
basicConstraints       = critical,CA:TRUE,pathlen:0
keyUsage               = critical,keyCertSign,cRLSign
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid:always
nameConstraints        = critical,permitted;DNS:tp.local,permitted;email:tp.local

[ ext_crl ]
authorityKeyIdentifier = keyid:always
```

B.2 `inter.cnf`

```
# inter.cnf - AC intermediaire : signe les certificats de serveur et de client
[ ca ]
default_ca = CA_inter

[ CA_inter ]
dir               = ./inter
database          = $dir/index.txt
new_certs_dir     = $dir/newcerts
serial            = $dir/serial
crlnumber         = $dir/crlnumber
certificate       = $dir/inter.crt
private_key       = $dir/inter.key
default_md        = sha256
default_days      = 90
default_crl_days  = 7
unique_subject    = no
policy            = policy_feuille
copy_extensions   = copy
crl_extensions    = ext_crl

[ policy_feuille ]
commonName = supplied

[ ext_serveur ]
basicConstraints       = critical,CA:FALSE
keyUsage               = critical,digitalSignature
extendedKeyUsage       = serverAuth
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid,issuer

[ ext_client ]
basicConstraints       = critical,CA:FALSE
keyUsage               = critical,digitalSignature
extendedKeyUsage       = clientAuth
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid,issuer

[ ext_crl ]
authorityKeyIdentifier = keyid:always
```

C TP 2 – Script `pki.sh`

```
#!/usr/bin/env bash
# pki.sh - construit la PKI du TP 2 (racine, intermediaire, serveur, clients)
```

```
# Usage : ./pki.sh            (refuse d'ecraser une PKI existante)
set -euo pipefail

SUJET_AC="/C=BF/O=Universite TP"
[ -e racine/racine.key ] && { echo "PKI deja presente : supprimez racine/ et inter/" >&2; exit
    1; }

# --- Arborescence des deux autorites ---------------------------------------
for ac in racine inter; do
  mkdir -p "$ac"/{newcerts,crl}
  : > "$ac/index.txt"
  echo 1000 > "$ac/serial"
  echo 1000 > "$ac/crlnumber"
done
umask 077   # les cles privees ne sont lisibles que par leur proprietaire

# --- Etape 2 : AC racine (P-384, 10 ans) -----------------------------------
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out racine/racine.key
openssl req -x509 -new -key racine/racine.key -sha384 -days 3650 \
  -subj "$SUJET_AC/CN=Racine TP" \
  -addext "basicConstraints=critical,CA:TRUE,pathlen:1" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" \
  -addext "subjectKeyIdentifier=hash" -out racine/racine.crt

# --- Etape 3 : AC intermediaire (P-256, 5 ans) -----------------------------
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out inter/inter.key
openssl req -new -key inter/inter.key -sha256 \
  -subj "$SUJET_AC/CN=Intermediaire TP" -out inter/inter.csr
openssl ca -batch -notext -config racine.cnf -extensions ext_inter -days 1825 \
  -in inter/inter.csr -out inter/inter.crt

# --- Etape 4 : certificats de feuille (P-256, 90 jours) --------------------
# emettre <nom_fichier> <CN> <SAN> <section d'extensions>
emettre() {
  openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out "$1.key"
  openssl req -new -key "$1.key" -subj "/CN=$2" \
    -addext "subjectAltName=$3" -out "$1.csr"
  openssl ca -batch -notext -config inter.cnf -extensions "$4" -days 90 \
    -in "$1.csr" -out "$1.crt"
}
emettre serveur "www.tp.local"   "DNS:www.tp.local,DNS:tp.local" ext_serveur
emettre alice   "alice@tp.local" "email:alice@tp.local"          ext_client
emettre bob     "bob@tp.local"   "email:bob@tp.local"            ext_client

# --- CRL initiales (vides) et fichier combine ------------------------------
openssl ca -config inter.cnf  -gencrl -out inter/crl/inter.crl
openssl ca -config racine.cnf -gencrl -out racine/crl/racine.crl
cat inter/crl/inter.crl racine/crl/racine.crl > crls.pem
echo "PKI creee : racine/racine.crt inter/inter.crt serveur.crt alice.crt bob.crt"
```

D TP 3 – Configuration nginx

```
server {
    listen 443 ssl;
    server_name www.tp.local;

    ssl_certificate     /etc/nginx/tls/serveur_chaine.crt;
    ssl_certificate_key /etc/nginx/tls/serveur.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers off;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-ECDSA-
    CHACHA20-POLY1305;
```

```
    ssl_ecdh_curve X25519:prime256v1:secp384r1;
    ssl_session_tickets off;

    # Authentification mutuelle des clients
    ssl_client_certificate /etc/nginx/tls/racine.crt;
    ssl_verify_client on;
    ssl_verify_depth 2;
    ssl_crl /etc/nginx/tls/crls.pem;

    add_header Strict-Transport-Security "max-age=31536000" always;

    location / {
        return 200 "Bonjour $ssl_client_s_dn\n";
    }
}
```

E TP 3 – Script `surveiller.py`

```
#!/usr/bin/env python3
"""surveiller.py - supervision des dates d'expiration des certificats.

Usage : surveiller.py [dossier]      (defaut : /etc/nginx/tls)
Code de sortie : 0 = tout OK, 1 = au moins une ALERTE, 2 = au moins un CRITIQUE
(compatible Nagios / Zabbix / Prometheus textfile / cron).
"""
import sys
from datetime import datetime, timezone
from pathlib import Path

from cryptography import x509

SEUIL_ALERTE, SEUIL_CRITIQUE = 30, 7
NIVEAUX = ("OK", "ALERTE", "CRITIQUE")


def fin_validite(cert: x509.Certificate) -> datetime:
    try:                                    # cryptography >= 42
        return cert.not_valid_after_utc
    except AttributeError:                  # anciennes versions
        return cert.not_valid_after.replace(tzinfo=timezone.utc)


def lire_certificats(chemin: Path):
    """Rend tous les certificats d'un fichier (PEM multi-certificats ou DER)."""
    donnees = chemin.read_bytes()
    if b"-----BEGIN CERTIFICATE-----" in donnees:
        return x509.load_pem_x509_certificates(donnees)
    return [x509.load_der_x509_certificate(donnees)]


def niveau(jours: int) -> int:
    if jours < SEUIL_CRITIQUE:              # inclut les certificats deja expires
        return 2
    if jours < SEUIL_ALERTE:
        return 1
    return 0


def main(dossier: str) -> int:
    maintenant = datetime.now(timezone.utc)
    pire = 0
    for chemin in sorted(Path(dossier).iterdir()):
```

```
        if chemin.suffix not in (".crt", ".pem", ".cer", ".der") or not chemin.is_file():
            continue
        try:
            certs = lire_certificats(chemin)
        except ValueError:                # cle privee, CRL, fichier illisible
            continue
        for c in certs:
            jours = (fin_validite(c) - maintenant).days
            n = niveau(jours)
            pire = max(pire, n)
            cn = c.subject.rfc4514_string()
            print(f"{NIVEAUX[n]:9} {jours:5d} j  {chemin.name}  [{cn}]")
    return pire


if __name__ == "__main__":
    sys.exit(main(sys.argv[1] if len(sys.argv) > 1 else "/etc/nginx/tls"))
```

F TP 3 – Script `renouveler.sh`

```
#!/usr/bin/env bash
# renouveler.sh - renouvellement du certificat du serveur (TP 3, etape 6)
# Variables : PKI (dossier de la PKI), TLS (dossier nginx), SEUIL (jours),
#             CN, SAN, FORCE=1 (renouveler meme si le seuil n'est pas atteint)
set -euo pipefail
umask 077

PKI=${PKI:-/opt/pki}
TLS=${TLS:-/etc/nginx/tls}
SEUIL=${SEUIL:-30}
CN=${CN:-www.tp.local}
SAN=${SAN:-DNS:www.tp.local,DNS:tp.local}
RELOAD_CMD=${RELOAD_CMD:-systemctl reload nginx}
DATE=$(date +%Y%m%dT%H%M%S)
ARCHIVE="$TLS/archive/$DATE"
log() { echo "$(date -Is) renouveler: $*"; }

# 0. Faut-il renouveler ? (-checkend rend 0 si le certificat reste valide SEUIL jours)
if [ "${FORCE:-0}" != "1" ] && \
   openssl x509 -checkend $((SEUIL * 86400)) -noout -in "$TLS/serveur_chaine.crt" >/dev/null;
    then
  log "certificat valide plus de $SEUIL jours : rien a faire"; exit 0
fi

TMP=$(mktemp -d); trap 'rm -rf "$TMP"' EXIT
cd "$PKI"

# 1. Nouvelle cle et nouvelle demande (la cle ne quitte jamais cette machine)
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out "$TMP/serveur.key"
openssl req -new -key "$TMP/serveur.key" -subj "/CN=$CN" \
  -addext "subjectAltName=$SAN" -out "$TMP/serveur.csr"

# 2. Signature par l'intermediaire
openssl ca -batch -notext -config inter.cnf -extensions ext_serveur -days 90 \
  -in "$TMP/serveur.csr" -out "$TMP/serveur.crt"
cat "$TMP/serveur.crt" inter/inter.crt > "$TMP/serveur_chaine.crt"

# 3. Verification AVANT tout deploiement : chaine, nom, usage, cle = certificat
openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt \
  -purpose sslserver -verify_hostname "$CN" "$TMP/serveur.crt" >/dev/null
[ "$(openssl pkey -in "$TMP/serveur.key" -pubout)" = \
  "$(openssl x509 -in "$TMP/serveur.crt" -pubkey -noout)" ] \
```

```
  || { log "ERREUR : la cle ne correspond pas au certificat"; exit 1; }
log "nouvelle chaine verifiee"

# 4a. Sauvegarde de l'ancien certificat et de l'ancienne cle (retour arriere)
mkdir -p "$ARCHIVE"
cp -p "$TLS/serveur.key" "$TLS/serveur_chaine.crt" "$ARCHIVE/"

retour_arriere() {
  log "ECHEC : retour a la version precedente ($ARCHIVE)"
  install -m 600 "$ARCHIVE/serveur.key" "$TLS/serveur.key"
  install -m 644 "$ARCHIVE/serveur_chaine.crt" "$TLS/serveur_chaine.crt"
  $RELOAD_CMD || true
  exit 1
}

# 4b. Installation, test de la configuration, rechargement sans coupure
install -m 600 "$TMP/serveur.key" "$TLS/serveur.key"
install -m 644 "$TMP/serveur_chaine.crt" "$TLS/serveur_chaine.crt"
nginx -t || retour_arriere
$RELOAD_CMD || retour_arriere

# 4c. Controle apres deploiement : le serveur presente bien le nouveau certificat
sleep 1
ATTENDU=$(openssl x509 -in "$TMP/serveur.crt" -noout -fingerprint -sha256)
SERVI=$(openssl s_client -connect "${CIBLE:-127.0.0.1:443}" -servername "$CN" </dev/null 2>/dev
    /null \
        | openssl x509 -noout -fingerprint -sha256) || SERVI=""
[ "$ATTENDU" = "$SERVI" ] || retour_arriere
log "OK : nouveau certificat en service ($(openssl x509 -in "$TMP/serveur.crt" -noout -enddate)
    )"
```